

Scala Interview Questions with Answers

BASIC LEVEL

Standard Functions

Q1. What is the difference between defining a function with and without an explicit return type?

With explicit return type, compiler enforces type. Without it, Scala infers automatically.

Q2. Why is it considered best practice to define explicit return types?

Prevents accidental type changes, improves readability, avoids recursive inference issues, makes APIs predictable.

Q3. What happens if a function does not return the declared type?

Compilation error due to Scala's type safety.

Q4. How does Scala infer return types?

Scala analyzes the last expression in function body and infers its type.

Q5. Difference between Unit and Double as return types?

Double returns a value, Unit represents no meaningful return value.

Q6. Why should pricing logic avoid printing inside calculation functions?

Printing is a side effect, making the function impure and harder to test.

Varargs

Q7. Syntax for varargs?

```
def calculateTotal(prices: Double*): Double
```

Q8. How are varargs represented internally?

As a Seq[Double].

Q9. How do you pass a sequence to a varargs function?

```
calculateTotal(list: _*)
```

Q10. Common mistakes with varargs?

Forgetting :_*, placing varargs not last, using mutable collections.

Q11. Why avoid mutable collections with varargs?

Maintains immutability and prevents unintended state modification.

Q12. Can a method have both regular parameters and varargs?

Yes, varargs must be last.

Lambda Expressions

Q13. Syntax of a lambda expression?

```
(x: Int) => x * 2
```

Q14. What does => represent?

Separates parameters from the function body.

Q15. Difference between lambda and normal method?

Lambda is anonymous function value; method is defined with def.

INTERMEDIATE LEVEL

Functional Transformations

Q16. Difference between map, filter, foreach?

map transforms, filter selects, foreach performs side effects.

Q17. Why is foreach not pure?

Used for side effects like printing.

Q18. How does immutability improve predictability?

Prevents shared-state bugs and race conditions.

Q19. What are side effects?

Any observable interaction outside function scope.

Q20. Why avoid external mutable state in filtering logic?

Breaks referential transparency.

Pure vs Impure Functions

Q21. Define a pure function.

No side effects; always returns same output for same input.

Q22. What makes a function impure?

Modifies state, performs IO, depends on external mutable data.

Q23. Why are pure functions easier to test?

They produce predictable results, no mocks needed.

Q24. Is logging a side effect?

Yes, interacts with external systems.

Q25. Can a function reading global variable be pure?

No, output depends on external mutable state.

Q26. Advantages of separating logic from side effects?

Easier testing, maintainable, cleaner architecture.

Q27. What is referential transparency?

Expression can be replaced with its value without changing behavior.

Higher-Order Functions

Q28. What is a Higher-Order Function?

Takes function as parameter or returns a function.

Q29. Function type syntax for Double => Double?

val fee: Double => Double

Q30. Why useful in payment strategies?

Allows flexible, interchangeable business logic.

Q31. Difference between passing method vs lambda?

Method must be converted to function value; lambda is already a function value.

Q32. How does Scala treat functions internally?

As instances of FunctionN traits.

Q33. What is partial application?

Fix some arguments of a function and return new function.

Q34. What is currying?

Transform multiple parameters into multiple parameter lists.

Q35. Why are HOFs important architecturally?

Enable abstraction, reuse, and functional composition.

ADVANCED LEVEL

Classes & Inheritance

Q36. Difference between class and trait?

Class is blueprint for objects; trait is reusable behavior mixin.

Q37. How does constructor parameter visibility work?

Parameters are private unless prefixed with val or var.

Q38. What does override enforce?

Ensures method actually overrides parent method.

Q39. What happens if override is omitted?

Compilation error.

Q40. Why is immutability important in class design?

Prevents unintended state changes.

Q41. What is runtime polymorphism?

Method resolution happens at runtime (overriding).

Method Overloading vs Overriding

Q42. What is method overloading?

Same name, different parameter lists (compile-time polymorphism).

Q43. What is method overriding?

Redefining parent method in child class (runtime polymorphism).

Q44. How does Scala choose overloaded method?

Based on parameter types at compile time.

Q45. How does dynamic dispatch work?

Method implementation chosen based on actual object type at runtime.

Q46. Can return types alone differentiate overloaded methods?

No, parameter list must differ.

Q47. What is compile-time polymorphism?

Resolved during compilation (overloading).

Q48. What is runtime polymorphism?

Resolved during execution (overriding).

Abstract Classes & Traits

Q49. When choose abstract class over trait?

When constructor parameters or shared state needed.

Q50. What is trait linearization?

Scala resolves trait methods right-to-left in mixin order.